

FACULTY OF ENGINEERING AND BASIC SCIENCES
ACADEMIC PROGRAM: DATA ENGINEERING AND ARTIFICIAL INTELLIGENCE

COURSE: ETL (G01)

Workshop-2: Building a Reliable Batch Data Pipeline

1. Workshop at a Glance

Item	Requirement
Scenario	Integrate Spotify data from CSV and Grammy Awards data from a relational source database.
Core product	A reliable batch pipeline and a trusted dimensional Data Warehouse.
Orchestration	Apache Airflow 3.1.8 using the TaskFlow API.
Automated validation	Great Expectations at raw and prepared-data gates.
Assessed databases	PostgreSQL or MySQL. Any alternative requires prior instructor approval. SQLite is not accepted.
Analytics	At least three KPIs and three visualizations querying the Data Warehouse.
Submission	An organized, reproducible GitHub repository.
Assessment	Engineering Solution / Repository: 80%; Presentation and Defense: 20%.

ENGINEERING PRINCIPLE A Data Engineer does not build scripts. A Data Engineer designs systems.

2. Purpose

Design, implement, validate, orchestrate, monitor, and document a reliable batch data pipeline that integrates two heterogeneous data sources and produces a trusted analytical Data Warehouse. This workshop evaluates an engineering system, not a collection of disconnected scripts. A complete solution must connect analytical requirements to required data, quality controls, transformations, dimensional structures, and analytical outputs.

KEY IDEA Reliable data pipelines do not assume that data is correct. They verify it.

IMPORTANT Reliable does not mean that nothing fails. Reliable means that failures are anticipated, controlled, and observable.

3. Learning and Engineering Outcomes

- Define analytical requirements that require information from both data sources.
- Profile both sources and identify evidence-based data quality risks.
- Convert profiling findings into explicit quality rules, metrics, thresholds, and severity levels.
- Implement automated validation with Great Expectations.
- Design raw-data and prepared-data validation gates.
- Apply justified cleaning, harmonization, integration, and transformation rules.
- Design and implement a requirements-driven dimensional Data Warehouse.
- Build an Airflow DAG that orchestrates extraction, validation, transformation, validation, and loading.
- Configure appropriate failure handling and selective retries.
- Demonstrate both successful execution and controlled failure behavior.
- Interpret task states and logs through the Airflow Web UI.
- Demonstrate safe reruns without unintended duplicate effects.
- Build KPIs and visualizations from the Data Warehouse.
- Document and defend the complete engineering solution.

4. Engineering Framework

4.1 End-to-End Reasoning Chain

Every important implementation decision must be traceable through this chain:

Stage	Engineering question
Business / analytical requirement	What decision or question must the data product support?
Required data	Which attributes and sources are necessary?
Data profiling	What does the evidence reveal about the data?
Quality risk	How could the evidence threaten the analytical requirement?
Quality dimension	Is the concern completeness, validity, uniqueness, consistency, timeliness, or another justified dimension?
Quality rule	What must be true?
Metric / threshold / severity	How is compliance measured, what is acceptable, and how serious is failure?
GX Expectation	How will the rule be automated?
Validation result	What evidence did the check produce?
Pipeline action	Should the pipeline continue, warn, quarantine, stop, or follow another documented response?

4.2 Reliable Pipeline Pattern

SOURCE	EXTRACT	RAW VALIDATION	TRANSFORM	PREPARED VALIDATION	LOAD	TARGET
Spotify CSV + Grammy DB	Acquire source data	Can data enter transformation?	Clean, standardize, integrate, derive	Is prepared data load-ready?	Write dimensional structures	Trusted DW

ENGINEERING PRINCIPLE Validation detects violations, measures quality, and provides evidence. Transformation applies known engineering rules to clean, standardize, harmonize, integrate, convert, and derive.

IMPORTANT Validation and transformation are not the same activity. Do not modify data only to make a validation pass.

4.3 Source Scenario

Source	Provided form	Pipeline extraction source	Required clarification
Spotify	CSV file	CSV file	The pipeline reads Spotify data from the provided file source.
Grammy Awards	CSV file	PostgreSQL or MySQL source database	Loading the provided CSV into this database is source preparation, not the final ETL Load stage.

REQUIRED EVIDENCE Show that the operational pipeline extracts Grammy data from the relational source database, not directly from the provided Grammy CSV.

5. Required Solution Architecture

The submitted architecture may use different task names, provided that it preserves both source branches, both validation gates, the integration dependency, the dimensional load, and the analytical target.

Spotify branch	Convergence and target	Grammy branch
SPOTIFY CSV		GRAMMY DATABASE
↓		↓
extract_spotify		extract_grammys
↓		↓
validate_spotify_raw	both validated branches converge	validate_grammys_raw
	↓	
	transform_and_integrate	
	↓	
	validate_prepared	
	↓	
	load_dw	
	↓	
	DATA WAREHOUSE	
	↓	

Spotify branch	Convergence and target	Grammy branch
	KPIs / DASHBOARD	

Quality design surrounding the flow	Reliability controls surrounding the flow
Profiling -> Quality Risks -> Quality Rules -> Great Expectations -> Validation Gates	Dependencies Selective retries Logs Monitoring Controlled failure Safe rerun

REQUIRED The complete workflow must be orchestrated by Apache Airflow 3.1.8. The DAG should use the TaskFlow API and the public authoring interface from airflow.sdk. PythonOperator is not required.

```
from airflow.sdk import dag, task
```

This import is a technology constraint, not a solution template. The team remains responsible for task design, interfaces, dependencies, validation behavior, and failure policy.

6. Work Requirements

6.1 Analytical Requirements and Solution Scope

Before profiling, validation, or dimensional modeling, define at least three analytical requirements that require information from both Spotify and Grammy data. Each requirement must be specific enough to determine which data, transformations, quality controls, dimensional structures, and analytical outputs are needed.

ID	Analytical Requirement	Required Data	Source(s)	Expected KPI(s)	Required Level of Detail

- State the analytical question or decision that each requirement supports.
- Identify the necessary attributes and the source of each attribute.
- Define the expected metric or KPI and the required level of detail.
- Explain why combining both sources is necessary.
- Maintain traceability from each requirement to downstream design decisions.

REQUIRED EVIDENCE Completed analytical-requirement table, a concise scope statement, and traceability references used consistently throughout the repository.

6.2 Environment and Source Preparation

Component	Required choice / constraint
Orchestrator	Apache Airflow 3.1.8 in Docker / Docker Compose.
DAG authoring	Python with the Airflow TaskFlow API.
Validation	Great Expectations; automated checks are mandatory.
Relational database	PostgreSQL or MySQL. Other databases require instructor approval. SQLite is not accepted.
BI tool	Power BI, Tableau, Looker Studio, or another approved BI tool.
Version control	Git and GitHub.

Prepare the Grammy source database by loading the provided Grammy CSV into a source table. Preserve the original input files and document the import process, table definition, row-count reconciliation, and any source-preparation decisions.

IMPORTANT Populating the Grammy source database creates the operational source for extraction. It is not the final ETL Load into the analytical Data Warehouse.

- [] Docker Compose environment starts reproducibly.
- [] Airflow reports version 3.1.8.
- [] Spotify CSV is available through the documented project path.
- [] Grammy source table exists in PostgreSQL or MySQL and reconciles with the provided CSV.
- [] Secrets are not committed; .env.example documents required variables without real credentials.
- [] Connection setup and execution commands are documented in README.md.

6.3 Data Profiling and Quality Risk Analysis

Profile both sources before defining cleaning or validation behavior. Profiling is evidence gathering: it describes the observed data and supports engineering decisions. It must not silently alter the data.

Evidence category	Minimum analysis
Structure	Schema, columns, row counts, data types, and fields required by the analytical requirements.
Completeness	Missing-value counts and percentages, interpreted in context.
Uniqueness and duplication	Candidate keys, duplicate patterns, and whether repeated values are expected at the source grain.
Categorical content	Distinct values, frequency patterns, unexpected labels, spelling/case variation, and domain risks.

Evidence category	Minimum analysis
Numerical content	Ranges, distributions, impossible values, suspicious extremes, and units where relevant.
Temporal content	Date coverage, parseability, granularity, invalid dates, and gaps relevant to the requirements.
Cross-source consistency	Potential compatibility problems affecting matching, comparison, or integration.

Dataset / Attribute	Profiling Evidence	Potential Quality Risk	Related Requirement

REQUIRED EVIDENCE A reproducible profiling notebook plus concise tables or figures that support every reported risk. Screenshots alone are not sufficient evidence.

6.4 Dimensional Model Design

Before designing the final transformation and integration logic, define the analytical target that the pipeline must produce.

The dimensional model must be derived from the analytical requirements and supported by the available source data.

At minimum, define:

- the business process;
- the grain of the fact table;
- the dimensions;
- the measures;
- primary and foreign keys;
- surrogate/business-key strategy;
- relationships;
- how each analytical requirement is supported.

Design decision	Required content
Business process	The process represented by the fact table.
Grain of the fact table	A precise statement of what one fact row represents.
Dimensions	Descriptive analysis contexts and their keys.
Measures	Numeric or countable facts consistent with the declared grain.
Keys and relationships	Primary keys, foreign keys, surrogate/business-key treatment, and referential constraints.
Requirement support	How facts, dimensions, and queries answer the defined analytical requirements.

6.5 Data Quality Rules and Validation Design

Convert evidence-based risks into explicit rules. Define at least six meaningful rules across the complete pipeline. Do not force an arbitrary equal number per dataset; coverage must follow the actual risks and analytical requirements.

Rule ID	Dataset / Layer	Attribute(s)	Quality Dimension	Quality Rule	Metric / Threshold	Severity	Related Requirement

Each rule must be justified by at least one of the following: profiling evidence, an analytical requirement, a documented source contract, or an expected post-transformation invariant.

Severity	Meaning	Policy expectation
Critical	The violation makes continued processing or loading unsafe.	Normally blocks the affected downstream path; the team must justify any exception.
Warning	The issue requires visibility but may not invalidate the batch.	Record and expose the result; continue only under a documented policy.
Informational	The result provides monitoring context or a trend signal.	Record for interpretation; it does not independently block processing.

KEY IDEA A threshold is an engineering decision. Explain why it is appropriate for the protected requirement and the observed data; do not choose it only because the current batch passes.

6.6 Great Expectations Design

Implement the approved quality rules with Great Expectations. Organize the validation design so that the team can explain which data object is validated, which Expectation Suite applies, how the Validation Definition connects them, and how a Checkpoint or equivalent controlled execution produces and records results.

Design element	Required documentation
Expectation	The GX check that implements a declared quality rule.
Expectation Suite	The coherent group of checks applied to a specific dataset or layer.
Validation Definition	The explicit association between the data definition and the suite.
Checkpoint / execution	How validation is run, results are retained, and policy is applied.
Validation result	Overall success plus useful per-rule evidence, not only a screenshot of a Boolean value.

REQUIRED EVIDENCE A mapping from every assessed GX Expectation to its Rule ID, plus machine-readable or clearly preserved validation results for successful and failed executions.

6.7 Data Extraction and Raw Validation

The final DAG must contain two independently observable source branches:

Spotify branch	Grammy branch
extract_spotify -> validate_spotify_raw	extract_grammys -> validate_grammys_raw

Each raw validation gate must answer: Are the incoming data safe enough to continue processing?

- Extract the source without hiding source-quality problems through premature cleaning.
- Apply Expectations derived from the documented rules for that raw source.
- Record interpretable validation results.
- Stop the affected downstream processing when a Critical rule fails.
- Apply the documented policy to Warning and Informational results.
- Keep raw outputs or metadata sufficient to investigate the evaluated batch.

REQUIRED EVIDENCE Successful raw-validation results for both sources and at least one controlled Critical raw-validation failure linked to its Rule ID.

6.8 Data Transformation and Integration

Transformation is a single engineering phase that may include cleaning, standardization, harmonization, type conversion, missing-value handling, duplicate handling, date normalization, derivation, and cross-source integration. Do not create a disconnected 'cleaning' stage merely to repair validation failures.

Decision	Required explanation
Rule and rationale	What known engineering rule is applied and what evidence or requirement justifies it?
Affected fields	Which columns or records are changed?
Before / after behavior	How is the resulting representation different?
Exception handling	What happens when a value cannot be transformed safely?
Analytical impact	Which requirement, measure, dimension, or integration step depends on the decision?

IMPORTANT Do not modify data only to make a validation pass. The transformation must be defensible independently of the current validation result.

Integration Contract

Item	Team decision and evidence
Integration key(s)	Identify the source attributes and the prepared matching key or strategy.
Cardinality	State and verify the expected one-to-one, one-to-many, many-to-one, or many-to-many behavior.
Preprocessing	Document normalization required before matching.

Item	Team decision and evidence
Unmatched records	Define how unmatched rows are measured and treated.
Duplicate matches	Define how ambiguous or repeated matches are detected and treated.
Assumptions and limitations	State what the integration can and cannot establish reliably.

KEY IDEA Integration is not adequately documented by the phrase 'perform a merge.' It is a data contract with keys, cardinality, exceptions, evidence, and analytical consequences.

6.9 Prepared Data Validation

After transformation and integration, run a second mandatory validation gate:

Input	Gate	Question	Output
Prepared integrated dataset	validate_prepared	Did transformation produce data suitable for loading?	Accepted prepared batch or controlled failure

- **Validate the expected prepared schema and required attributes.**
- **Check valid ranges and allowed domains that should hold after transformation.**
- **Verify integration integrity, including relevant unmatched or duplicate-match indicators.**
- **Reject critical invalid values that would compromise the dimensional load.**
- **Verify analytical readiness for the defined requirements.**

Gate	Primary question
Raw validation	Can these data safely enter transformation?
Prepared validation	Did transformation produce acceptable data for loading?

REQUIRED EVIDENCE A distinct prepared-data Expectation Suite or clearly separated rule set, its execution result, and proof that load_dw is gated by its Critical outcome.

6.10 Data Warehouse Loading and Dimensional Modeling

Implement the star schema in PostgreSQL or MySQL. The dimensional model is mandatory and must be driven by the analytical requirements.

IMPORTANT The final ETL Load writes validated prepared data into the dimensional Data Warehouse. It is different from the earlier Grammy source-preparation import.

6.11 Workflow Orchestration and Reliability

The Airflow DAG must expose the complete engineering workflow rather than hiding the pipeline inside one task. Task boundaries should support meaningful states, logs, failure isolation, and reruns.

Area	Minimum requirement
Tasks	Represent both extractions, both raw validations, transformation/integration, prepared validation, and DW load.
Dependencies	Prevent transformation until both required raw branches satisfy the policy; prevent load until prepared validation succeeds.
Task interfaces	Pass references, paths, identifiers, or compact metadata; do not use the orchestrator metadata store as a bulk-data transport.
Failure conditions	Make deterministic data-quality failures explicit and observable.
Retries	Apply selectively to transient operational failures, not indiscriminately.
Logging	Record batch context, material row counts, validation outcomes, load results, and actionable failure causes.
Execution evidence	Preserve successful, controlled-failure, and rerun evidence.

ENGINEERING PRINCIPLE A DAG is an executable dependency and control model. A visually connected set of tasks is not sufficient unless the states, policies, and evidence reflect the intended system behavior.

Failure Policy

Condition	Severity / Type	Pipeline Response	Retry?	Justification

Failure class	Examples	Expected retry reasoning
Transient operational	Temporary database outage; temporary network interruption.	A bounded retry may be appropriate because another attempt can succeed without changing the input or code.
Deterministic data / contract	Missing required column; invalid schema; Critical GX failure.	Automatic repetition normally reproduces the same failure. Stop and surface evidence unless a different documented recovery exists.

IMPORTANT A blanket policy such as retries=3 everywhere is not evidence of reliability. Justify retry count, delay, and scope for each task class where retries are used.

Logging and Observability

Use Airflow task states, timestamps, and logs to answer:

Question	Expected evidence
What ran?	DAG Run and task identifiers, batch or source context.
When?	Logical/run time and relevant task timestamps.
Did it succeed?	DAG and Task Instance states plus material completion evidence.
Where did it fail?	The failed task and blocked/skipped downstream behavior.
Why?	Actionable log message and linked validation or operational evidence.
What happened next?	Retry, stop, warning continuation, or recovery behavior according to policy.

7. Mandatory Reliability Tests

7.1 Test A - Successful Run

Execute the full pipeline with an input batch that satisfies the Critical policy.

Stage	Expected conceptual result
Extract both sources	Success
Raw validation gates	Success under the documented policy
Transform and integrate	Success with recorded reconciliation evidence
Prepared validation	Success

Stage	Expected conceptual result
Load	Success with row-count or load-summary evidence
Analytics	Queries and dashboard reflect DW data

7.2 Test B - Controlled Critical Quality Failure

Intentionally introduce or activate a condition that violates one justified Critical raw-data rule. The team chooses the condition; this brief does not prescribe a specific invalid value.

Stage	Expected conceptual result
Extract affected source	Success
Affected raw validation	Failure
Downstream transformation	Does not proceed normally
Prepared validation and load	Do not proceed normally
Evidence	Failure remains visible and diagnosable

REQUIRED EVIDENCE Airflow Graph or Grid view, failed task state, GX validation result, failed task log, and a written explanation of the blocking rule and downstream behavior.

7.3 Repeatability and Safe Rerun

Run the pipeline again using the same input batch. Demonstrate that the rerun does not silently create unintended duplicate effects in the Data Warehouse.

- State the chosen strategy, such as replace, truncate-and-load, controlled upsert, business-key enforcement, batch-key management, or another justified approach.
- Show the relevant keys, constraints, commands, or load logic.
- Compare target row counts and selected measures before and after the rerun.
- Explain what happens after a partially completed or failed run.

ENGINEERING PRINCIPLE A rerun must not silently corrupt the analytical target.

4.4 Reliability Evidence Register

Evidence ID	Run / Task	Artifact or Path	What It Proves	Related Policy / Rule

Evidence ID	Run / Task	Artifact or Path	What It Proves	Related Policy / Rule

8. Analytics, Traceability, and Evidence

8.1 Data Warehouse Analytics

Build at least three KPIs and/or three analytical visualizations. Every output must answer or support a declared analytical requirement and must query the dimensional Data Warehouse. Direct CSV connections are not accepted as the assessed dashboard solution.

Analytical Requirement	DW Element / Query	KPI / Visualization

KEY IDEA The dashboard should answer the analytical requirements using trusted data from the Data Warehouse. Decorative complexity is not an evaluation objective.

8.2 End-to-End Traceability Matrix

Provide enough representative rows to demonstrate the complete design chain. Do not create dozens of repetitive rows; select material requirements and rules that show genuine end-to-end traceability.

Requirement	Required Data	Quality Risk	DQ Rule	GX Expectation	Transformation	DW Element	KPI / Visualization

8.3 Minimum Evidence Set

- [] Analytical-requirement table and solution scope.
- [] Conceptual architecture and implemented DAG structure.
- [] Profiling notebook and evidence for both sources.
- [] Quality-risk analysis and quality-rule table.
- [] GX design and validation outputs for raw and prepared layers.
- [] Transformation and integration decision record.
- [] Dimensional model and executable schema definition.
- [] Successful DAG Run evidence.

- [] Controlled failed DAG Run evidence, including failed task logs.
- [] Selective-retry evidence or a justified explanation of tasks configured without retry.
- [] Safe-rerun evidence.
- [] Dashboard connection to the DW and requirement-to-KPI traceability.

IMPORTANT Screenshots must be legible, cropped to the relevant evidence, and accompanied by interpretation. A screenshot without a claim and explanation is weak engineering evidence.

9. Deliverables in GitHub

9.1 Suggested Repository Structure

The following structure is suggested, not mandatory. An alternative is acceptable when it preserves separation of concerns and remains reproducible.

```
workshop-2/
|-- dags/
|   |-- reliable_music_pipeline.py
|-- src/
|   |-- extract.py
|   |-- transform.py
|   |-- validation.py
|   |-- load.py
|-- gx/
|-- notebooks/
|   |-- data_profiling.ipynb
|-- sql/
|   |-- source_setup.sql
|   |-- dw_schema.sql
|-- docs/
|   |-- architecture.*
|   |-- quality_rules.*
|   |-- evidence/
|-- requirements.txt
|-- docker-compose.yaml
|-- .env
|-- .gitignore
`-- README.md
```

ENGINEERING PRINCIPLE Separate orchestration logic from reusable extraction, transformation, validation, and loading logic when practical. The DAG should communicate workflow structure rather than become a monolithic processing script.

Deliverable	Required contents
1. Profiling and Quality Design	Profiling notebook; profiling evidence; quality risks; quality-rule table; links to analytical requirements.
2. ETL and Data Integration	Extraction logic; transformation and integration logic; reconciliation evidence; documented assumptions and limitations.
3. Automated Data Validation	GX configuration; Expectations and Suites; Validation Definitions / execution design; raw and prepared validation results.
4. Reliable Airflow Pipeline	DAG; dependencies; selective retries; failure handling; logs; successful run; controlled failed run; safe-rerun evidence.
5. Analytical Data Product	Dimensional DW; star schema; KPI queries; dashboard connected to the DW.
6. Engineering Repository	Organized GitHub repository; reproducible setup; README; diagrams; documentation; evidence register.

9.2 README Requirements

- Problem and analytical objective
- Analytical requirements
- Data sources
- Pipeline architecture
- Data profiling findings
- Quality risks and quality rules
- Great Expectations validation design
- Transformation and integration strategy
- Dimensional model
- Airflow DAG design
- Failure and retry policy
- Successful execution evidence
- Controlled failure evidence
- Repeatability strategy
- Dashboard and analytical outputs
- Setup and execution instructions
- Assumptions and limitations

REQUIRED A missing mandatory validation gate, controlled failure test, safe-rerun demonstration, or DW-backed dashboard represents a missing system capability, not merely a documentation omission.

10. Evaluation

10.1 Engineering Solution / Repository - 4.00 points (80%)

Criterion	Description	Points
Analytical Requirements and Architecture	Requirements, required data, architecture, and end-to-end traceability.	0.40
Data Profiling and Quality Risk Analysis	Evidence-based profiling, risks, dimensions, rules, metrics, and thresholds.	0.60
ETL Transformation and Integration	Correct extraction plus justified cleaning, harmonization, integration, and reconciliation.	0.60
Automated Validation with Great Expectations	Raw and prepared validation, meaningful Expectations, results, and operational policy.	0.80
Airflow Orchestration and Reliability	DAG, dependencies, failure handling, selective retries, logs, controlled failure, and repeatability.	0.90
Dimensional DW and Analytics	Requirements-driven star schema, KPI queries, and dashboard sourced from the DW.	0.40
Repository and Documentation	Structure, reproducibility, README, evidence, security hygiene, and code quality.	0.30
TOTAL		4.00

10.2 Presentation and Defense - 1.00 point (20%)

Criterion	Points
Clear explanation of architecture and engineering decisions	0.35
Demonstration of successful and controlled failed runs	0.30
Technical understanding during questions	0.25
Communication and professionalism	0.10
TOTAL	1.00